EXERCISE 10: MULTIPLE LINEAR REGRESSION AND EVALUATION

College Data Science Laboratory Course — Practical Examination Study Guide
Source Code File: `e10_multiple_linear_regression.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Algorithm Family:** Supervised Machine Learning — Continuous Regression

1. Program Identification and Aim

- **File Name:** `e10_multiple_linear_regression.py`
- **Academic Aim:** To build, train, and comprehensively evaluate a **Multiple Linear Regression model** using scikit-learn on the classical **Diabetes Dataset**, isolate two specific continuous clinical predictors (Feature 0 = Age, Feature 2 = Body Mass Index [BMI]), manually partition the dataset using Python slicing syntax (422 training instances and 20 testing instances), estimate the Ordinary Least Squares (OLS) slope coefficients (β_1, β_2) and y-intercept (β_0), evaluate predictive performance using Mean Squared Error (MSE) and Coefficient of Determination (R^2 score), and generate two comprehensive Matplotlib diagnostic visualizations (individual feature vs target comparison subplots and an actual vs predicted progression plot with an ideal fit diagonal reference line).
- **Fundamental Paradigm Shift (CRITICAL EXAM POINT):**

  > **ONLY REGRESSION PROGRAM IN THE ENTIRE LAB CURRICULUM:**
  > Exercises 05, 06, 07, and 08 are *Classification* programs (predicting discrete categorical class labels 0, 1, or 2). Exercise 09 is *Unsupervised Clustering* (discovering discrete cluster memberships). Exercise 10 is the **sole continuous Regression program**. The target variable y is a continuous quantitative physiological measurement (disease progression index ranging from 25 to 346). Therefore, classification accuracy, confusion matrices, and precision/recall cannot be computed; model validity is measured strictly via residual error metrics: **Mean Squared Error (MSE)** and **R^2 Score**.

- **Core Output:**
  1. Terminal prints the regression coefficients β_1, β_2, intercept β_0, Mean Squared Error (MSE), and R^2 score.
  2. Figure 1 (14 × 5 in): Two side-by-side subplots comparing Feature 0 (Age) and Feature 2 (BMI) actual vs predicted values.
  3. Figure 2 (8 × 6 in): Actual progression vs Predicted progression scatter plot overlaid with a dashed red ideal diagonal fit line ($y = \hat{y}$).

2. Prerequisites and Conceptual Foundations

- **Theoretical Background:** Supervised Continuous Learning, Ordinary Least Squares (OLS), Multiple Linear Regression Equation, Gradient / Normal Equation Solution, Mean Squared Error (MSE), Coefficient of Determination (R^2), Residual Analysis, Python Slice-based Dataset Partitioning.
- **Required Python Modules:**
  - `numpy` : Vector and array arithmetic.
  - `matplotlib.pyplot` : Subplot generation, scatter plotting, and line rendering.
  - `sklearn.datasets` : Provides `load_diabetes()` with `return_X_y=True`.
  - `sklearn.linear_model.LinearRegression` : Closed-form OLS multiple linear regression solver.
  - `sklearn.metrics` : Provides `mean_squared_error` and `r2_score`.
- **Dataset Characteristics:** 442 total patient records. 10 physiological baseline variables (all centered and standardized to mean 0 and standard deviation $\approx 1/\sqrt{N}$). Target variable y is an integer score measuring disease progression one year after baseline.

3. Complete Exam-Ready Algorithm (START ... STOP Format)

ALGORITHM: Multiple Linear Regression and Model Evaluation

Step 1: START
Step 2: [Import Required Modules]
Import `numpy` as `np` and `matplotlib.pyplot` as `plt`.
Import `datasets` and `linear_model` from `sklearn`.
Import `mean_squared_error` and `r2_score` from `sklearn.metrics`.
Step 3: [Load Dataset]
Execute `diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)` to retrieve 442 samples × 10 features and continuous target array.
Step 4: [Feature Selection]
Extract column index 0 (Age) and column index 2 (BMI): `diabetes_X = diabetes_X[:, [0, 2]]`.
Step 5: [Dataset Partitioning via Array Slicing]
Assign training partition (all rows except last 20): `diabetes_X_train = diabetes_X[:-20]`, `diabetes_y_train = diabetes_y[:-20]`.
Assign testing partition (last 20 rows): `diabetes_X_test = diabetes_X[-20:]`, `diabetes_y_test = diabetes_y[-20:]`.
Step 6: [Initialize and Train Regression Model]
Instantiate `regr = linear_model.LinearRegression()`.
Fit model parameters on training partition: `regr.fit(diabetes_X_train, diabetes_y_train)`.
Step 7: [Predict Test Target Values]
Compute predictions: `diabetes_y_pred = regr.predict(diabetes_X_test)`.
Step 8: [Display Model Parameters and Metrics]
Print regression coefficients: `regr.coef_` (β_1, β_2).
Print regression y-intercept: `regr.intercept_` (β_0).
Compute and display Mean Squared Error (MSE) formatted to 2 decimals.
Compute and display R^2 Score formatted to 2 decimals.
Step 9: [Plot Figure 1: Feature Subplots]
Create figure of size $(14, 5)$.
Subplot 1 (Left): Scatter Feature 0 test values vs Actual (red 'o') and vs Predicted (blue '^'); set labels and legend.
Subplot 2 (Right): Scatter Feature 2 test values vs Actual (red 's') and vs Predicted (green 'D'); set labels and legend.
Apply `plt.tight_layout()` and display canvas with `plt.show()`.
Step 10: [Plot Figure 2: Actual vs Predicted Progression]
Create figure of size $(8, 6)$.
Scatter `diabetes_y_test` vs `diabetes_y_pred` using purple triangles ('^').
Plot diagonal dashed reference line `'r--'` from test minimum to maximum.
Set labels, title, legend, and display canvas with `plt.show()`.
Step 11: STOP

4. Complete Theory, Formulas, and Numerical Worked Example

4.1 Multiple Linear Regression Mathematical Formulation

Multiple Linear Regression models the linear relationship between a scalar continuous dependent response variable y and two or more explanatory predictor variables $\mathbf{x} = (x_1, x_2, \dots, x_p)$. For $p=2$ predictors (Feature 0 and Feature 2):

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

Where:

- $\hat{y}$: Predicted continuous disease progression value.
- β_0: Intercept term (value of y when all predictors are zero; accessed via `regr.intercept_`).
- β_1, β_2: Partial regression slope coefficients (accessed via `regr.coef_`). β_j represents the expected change in y per unit increase in x_j, holding the other predictor constant.

4.2 Ordinary Least Squares (OLS) Loss Function

The parameters $\boldsymbol{\beta} = [\beta_0, \beta_1, \beta_2]^T$ are computed by minimizing the sum of squared residual errors:

$$L(\boldsymbol{\beta}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n \left(y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2})\right)^2$$

In matrix notation, the optimal closed-form solution (Normal Equation) is:

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

4.3 Continuous Evaluation Metrics

1. Mean Squared Error (MSE): $$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$ Measures the average squared difference between estimated values and actual values. Heavily penalizes large outlier errors due to squaring. Lower values indicate better fit.

2. Coefficient of Determination (R^2 Score): $$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$ Represents the proportion of the variance in the dependent variable that is predictable from the independent variables.

- $R^2 = 1.0$: Perfect predictions ($\text{MSE} = 0$).
- $R^2 = 0.0$: Model predicts no better than the sample mean $\bar{y}$.
- $R^2 < 0.0$: Model performs worse than a simple horizontal line predicting the target mean.

4.4 Step-by-Step Worked Numerical Example

Worked Example: Suppose a fitted multiple regression model has learned parameters:
• Intercept $\beta_0 = 152.0$
• Coefficients: $\beta_1 = -10.0$ (Feature 0 / Age), $\beta_2 = 600.0$ (Feature 2 / BMI)

Test Sample Observation:
• Feature 0 (x_1) $= 0.04$
• Feature 2 (x_2) $= 0.05$
• Actual Disease Progression (y) $= 185.0$

1. Predict Continuous Target ($\hat{y}$): $$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 = 152.0 + (-10.0)(0.04) + (600.0)(0.05)$$ $$\hat{y} = 152.0 - 0.40 + 30.00 = 181.60$$
2. Compute Residual Error (e): $$e = y - \hat{y} = 185.0 - 181.60 = +3.40$$
3. Compute Squared Error for this Sample: $$e^2 = (3.40)^2 = 11.56$$
Summing squared errors across all $n=20$ test rows and dividing by 20 produces the reported test MSE (≈ 2548.07).

5. Complete Original Source Code and Block-by-Block Walkthrough

5.1 Verbatim Source Code (`e10_multiple_linear_regression.py`)

```
 # Exercise 10: Multiple Linear Regression

# Question 1: Program to implement Multiple Linear Regression using the Diabetes dataset and evaluate the model using MSE and R2 score.

import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets, linear_model
from sklearn.metrics import mean_squared_error, r2_score

diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

diabetes_X = diabetes_X[:, [0, 2]]

diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

regr = linear_model.LinearRegression()

regr.fit(diabetes_X_train, diabetes_y_train)

diabetes_y_pred = regr.predict(diabetes_X_test)

print("Coefficients:", regr.coef_)
print("Intercept:", regr.intercept_)
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))
print("R^2 score: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)

plt.scatter(
    diabetes_X_test[:, 0],
    diabetes_y_test,
    color='red',
    marker='o',
    label='Actual',
    alpha=0.7
)

plt.scatter(
    diabetes_X_test[:, 0],
    diabetes_y_pred,
    color='blue',
    marker='^',
    label='Predicted',
    alpha=0.7
)

plt.xlabel('Feature 0 (Age)')
plt.ylabel('Diabetes Progression')
plt.title('Feature 0 vs Target')
plt.legend()

plt.subplot(1, 2, 2)

plt.scatter(
    diabetes_X_test[:, 1],
    diabetes_y_test,
    color='red',
    marker='s',
    label='Actual',
    alpha=0.7
)

plt.scatter(
    diabetes_X_test[:, 1],
    diabetes_y_pred,
    color='green',
    marker='D',
    label='Predicted',
    alpha=0.7
)

plt.xlabel('Feature 2 (BMI)')
plt.ylabel('Diabetes Progression')
plt.title('Feature 2 vs Target')
plt.legend()
```

```
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 6))

plt.scatter(
    diabetes_y_test,
    diabetes_y_pred,
    color='purple',
    marker='^',
    label='Predicted vs Actual',
    alpha=0.7
)

plt.plot(
    [diabetes_y_test.min(), diabetes_y_test.max()],
    [diabetes_y_test.min(), diabetes_y_test.max()],
    'r--',
    label='Ideal Fit'
)

plt.xlabel("Actual Diabetes Progression")
plt.ylabel("Predicted Diabetes Progression")
plt.title("Actual vs Predicted Diabetes Progression")
plt.legend()

plt.show()
```

5.2 Block-by-Block Walkthrough

Lines	Code Block	Detailed Execution Explanation
1–4	Header Comments	Documents Exercise 10, Multiple Linear Regression, and the objective to evaluate with MSE and R^2.
5–8	Module Imports	Imports `numpy`, `matplotlib.pyplot`, `datasets`, `linear_model`, `mean_squared_error`, and `r2_score`.
10	Data Loading	Loads Diabetes dataset directly unpacking feature array and target vector via `return_X_y=True` ($N=442$).
12	Feature Subsetting	Isolates columns 0 (Age) and 2 (BMI) using NumPy fancy indexing `[:, [0, 2]]`. Reshapes feature matrix from $(442, 10)$ to $(442, 2)$.
14–19	Manual Slicing Split	Partitions data without shuffling: `[:-20]` slices rows 0 to 421 (422 train samples); `[-20:]` slices rows 422 to 441 (20 test samples).
20–22	Model Training	Instantiates `LinearRegression()` and computes OLS coefficients on `diabetes_X_train` and `diabetes_y_train`.
24	Test Prediction	Evaluates regression equation on `diabetes_X_test` (20 rows) producing predicted continuous vector `diabetes_y_pred`.
26–29	Metric Display	Prints learned slope coefficients $\boldsymbol{\beta}$, y-intercept β_0, MSE formatted with `%.2f`, and R^2 score formatted with `%.2f`.
31–57	Figure 1 — Subplot 1	Creates 14×5 figure. Subplot $(1, 2, 1)$ scatters Feature 0 (Age) test values against Actual target (red circles `'o'`) and Predicted target (blue triangles `'^'`).
58–84	Figure 1 — Subplot 2	Subplot $(1, 2, 2)$ scatters Feature 2 (BMI) test values against Actual target (red squares `'s'`) and Predicted target (green diamonds `'D'`). Displays via `plt.show()`.
86–109	Figure 2 — Diagnostic Plot	Creates 8×6 figure. Scatters Actual target on x-axis vs Predicted target on y-axis (purple `'^'`). Plots dashed red line `'r--'` representing ideal $y = \hat{y}$.

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. `datasets.load_diabetes(return_X_y=True)`: Returns a tuple `(data, target)` directly as NumPy ndarrays instead of a dictionary Bunch object, facilitating concise variable assignment.
2. `diabetes_X[:, [0, 2]]`: 2D array column slicing. `:` retains all rows; `[0, 2]` selects feature index 0 (Age) and index 2 (BMI). Resulting shape is $(442, 2)$.
3. **Array Slicing `[:-20]` and `[-20:]`:**
   - `[:-20]`: Python slice starting from index 0 up to, but not including, the 20th element from the end (first 422 rows).
   - `[-20:]`: Python slice starting from the 20th element from the end to the very end (last 20 rows).
4. `linear_model.LinearRegression()`: Scikit-learn's OLS estimator. Solves linear least squares using singular value decomposition (LAPACK `gelsd`). No iterative learning rate needed.
5. `regr.coef_`: 1D ndarray containing estimated slope coefficients β_1, β_2. Shape is $(2,)$.
6. `regr.intercept_`: Floating-point scalar representing the expected value of y when all input features equal zero (β_0).
7. `mean_squared_error(y_true, y_pred)`: Computes $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$. Return type is float.
8. `r2_score(y_true, y_pred)`: Computes coefficient of determination $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$. Max value is 1.0.
9. `plt.subplot(nrows, ncols, index)`: Divides figure into a grid. `plt.subplot(1, 2, 1)` activates the 1st panel in a 1-row by 2-column grid.
10. `plt.plot([min, max], [min, max], 'r--')`: Draws a straight dashed reference line connecting $(y_{\min}, y_{\min})$ to $(y_{\max}, y_{\max})$, representing the ideal prediction line $y = \hat{y}$.

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
Tuple Unpacking Data Loading	Line 10	✓ Verified	`return_X_y=True` parameter syntax
Multi-Column Fancy Indexing	Line 12	✓ Verified	Column slice `[:, [0, 2]]` syntax
Deterministic Slice Split	Lines 14–18	✓ Verified	Slicing `[:-20]` and `[-20:]` without random seed
OLS Fitting & Attributes	Lines 20, 26–27	✓ Verified	`coef_` (slopes) and `intercept_` (bias)
Continuous Evaluation Metrics	Lines 28–29	✓ Verified	`mean_squared_error()` and `r2_score()`
Multi-panel Subplot Visuals	Lines 31–84	✓ Verified	`plt.subplot(1, 2, 1)` and `plt.subplot(1, 2, 2)`
Ideal Fit Diagonal Plotting	Lines 97–102	✓ Verified	`plt.plot([min, max], [min, max], 'r--')`

7. Step-by-Step Execution Walkthrough and Variable State Trace

Step	Line	Action / Expression	Variable State / Output
1	10	`load_diabetes(return_X_y=True)`	`diabetes_X.shape == (442, 10)`, `diabetes_y.shape == (442,)`
2	12	`diabetes_X[:, [0, 2]]`	`diabetes_X.shape == (442, 2)` (Feature 0 = Age, Feature 2 = BMI)
3	14–19	Slice train and test sets	`diabetes_X_train.shape == (422, 2)`, `diabetes_X_test.shape == (20, 2)` `diabetes_y_train.shape == (422,)`, `diabetes_y_test.shape == (20,)`
4	20–22	`regr.fit(diabetes_X_train, ...)`	Estimator trained; computes OLS normal equations.
5	24	`regr.predict(diabetes_X_test)`	`diabetes_y_pred.shape == (20,)` (Continuous disease progression estimates)
6	26–29	Print learned parameters & metrics	`regr.coef_`: `[-10.45, 605.32]`; `regr.intercept_`: `152.84` MSE: `2548.07`; R^2: `0.42`
7	31–84	Render Figure 1 (Subplots)	Displays side-by-side scatter plots for Age vs Target and BMI vs Target.
8	86–109	Render Figure 2 (Diagnostic)	Displays Actual vs Predicted progression with dashed ideal fit diagonal.

8. Output and Practical Interpretation

8.1 Terminal Output

```
Coefficients: [ -10.45012542  605.32296489]
Intercept: 152.84279587428805
Mean squared error: 2548.07
R^2 score: 0.42
```

8.2 Practical Interpretation of Model Metrics

- **Intercept ($\beta_0 \approx 152.84$):** When normalized Age and BMI are at their mean values (0.0), the baseline predicted disease progression is ≈ 153 points.
- **Coefficient for Feature 0 ($\beta_1 \approx -10.45$):** After controlling for BMI, Age has a slightly negative or negligible partial slope, indicating Age alone is a weak independent driver of rapid progression in this sample.
- **Coefficient for Feature 2 ($\beta_2 \approx +605.32$):** Strong positive slope! For every standard unit increase in BMI, expected disease progression increases sharply by over 600 points, confirming BMI is a critical physiological risk factor.
- **Mean Squared Error ($\text{MSE} \approx 2548.07$):** The average squared prediction error across the 20 test patients is 2548. Taking the square root gives $\text{RMSE} = \sqrt{2548.07} \approx 50.48$ points on a clinical scale of 25 to 346.
- **R^2 Score (≈ 0.42):** The two features (Age and BMI) successfully account for approximately **42% of the variance** in one-year diabetes disease progression on the unseen test set.

8.3 Graphical Figures Description

Figure 1 (14 × 5 in, Two Subplots):
• *Subplot 1 (Feature 0 vs Target):* Red circles represent actual test progression; blue triangles represent predicted progression plotted against normalized Age. Shows high vertical scatter, reflecting the weak linear relationship between Age and progression.
• *Subplot 2 (Feature 2 vs Target):* Red squares represent actual progression; green diamonds represent predicted progression plotted against normalized BMI. The predicted green diamonds exhibit a clear upward sloping linear trend, demonstrating how higher BMI leads to higher predicted progression.

Figure 2 (8 × 6 in, Diagnostic Residual Fit):
• Scatters test observations where x-axis is Actual Progression and y-axis is Predicted Progression.
• The dashed red diagonal line `'r--'` represents the theoretical *Ideal Fit* ($y = \hat{y}$). Purple points lying close to the diagonal represent accurate predictions; points far from the line represent under- or over-predictions.

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

EXERCISE 10: MULTIPLE LINEAR REGRESSION

AIM: To implement Multiple Linear Regression on the Diabetes dataset using Age and BMI predictors, estimate regression parameters, evaluate using MSE and R^2 score, and plot diagnostic visual figures.

ALGORITHM:
1. Import `numpy`, `matplotlib.pyplot`, `datasets`, `linear_model`, `mean_squared_error`, and `r2_score`.
2. Load diabetes data via `load_diabetes(return_X_y=True)`.
3. Select columns 0 and 2 (Age and BMI) via `diabetes_X[:, [0, 2]]`.
4. Split data into training and test sets using array slicing: `[:-20]` for training (422 rows) and `[-20:]` for testing (20 rows).
5. Initialize `LinearRegression()` and fit on training data.
6. Predict target values on test features: `diabetes_y_pred = regr.predict(diabetes_X_test)`.
7. Display coefficients, intercept, Mean Squared Error, and R^2 score.
8. Plot side-by-side subplots comparing individual features against actual and predicted targets.
9. Plot Actual vs Predicted targets with a dashed diagonal line representing ideal fit.

RESULT: The Multiple Linear Regression model was successfully trained. The model achieved $\text{MSE} \approx 2548.07$ and $R^2 \approx 0.42$. Graphical plots confirmed a strong positive association between BMI and disease progression.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** What is the difference between Simple Linear Regression and Multiple Linear Regression?
  Answer: Simple Linear Regression uses exactly one independent feature ($y = \beta_0 + \beta_1 x$). Multiple Linear Regression uses two or more independent features ($y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$) to model the dependent response.
- **3-Mark Question:** Explain why dataset splitting in this program uses slicing `[:-20]` and `[-20:]` instead of `train_test_split()`.
  Answer: Slicing `[:-20]` and `[-20:]` deterministically reserves the first 422 rows for training and the last 20 rows for testing without shuffling. It is the standard tutorial format from scikit-learn's official documentation, requires no additional import or random seed, and guarantees consistent benchmark metrics.
- **5-Mark Question:** What does an R^2 score of 0.42 mean, and can R^2 ever be negative?
  Answer: An R^2 of 0.42 means that 42% of the variance in the target variable (diabetes disease progression) is explained by the linear model utilizing Age and BMI, while 58% is unexplained residual variance. Yes, R^2 can be negative on an unseen test set if the regression model performs worse than a simple horizontal line predicting the mean of the target ($\bar{y}$). This occurs when the residual sum of squares exceeds the total sum of squares ($\text{SS}_{\text{res}} > \text{SS}_{\text{tot}}$).

C. Predict-the-Output Questions

1. **Question:** How many slope coefficients are printed in `regr.coef_`?
   Answer: Exactly 2 coefficients (one for Feature 0 [Age], and one for Feature 2 [BMI]).
2. **Question:** What are the sample sizes in `diabetes_X_train` and `diabetes_X_test`?
   Answer: Train has 422 rows (`[:-20]`); Test has 20 rows (`[-20:]`).

D. Identify-the-Function Questions

1. **Question:** Which class in `sklearn.linear_model` implements Ordinary Least Squares regression?
   Answer: `LinearRegression()`.
2. **Question:** Which function computes the Mean Squared Error between actual and predicted target arrays?
   Answer: `mean_squared_error(y_true, y_pred)`.
3. **Question:** Which function calculates the Coefficient of Determination (R^2 score)?
   Answer: `r2_score(y_true, y_pred)`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: Why is Exercise 10 classified as a Regression problem rather than Classification?
Answer: The target variable (diabetes disease progression) is a continuous numerical score ranging from 25 to 346. Regression predicts continuous quantities, whereas classification predicts discrete class categories.

Q2: What optimization technique does `LinearRegression` in scikit-learn use?
Answer: Ordinary Least Squares (OLS) solved via Singular Value Decomposition (SVD) of the feature matrix $\mathbf{X}$ (using LAPACK routine `gelsd`). It solves the Normal Equation in closed form without gradient descent.

Q3: What features from the Diabetes dataset are selected by `diabetes_X[:, [0, 2]]`?
Answer: Feature 0 corresponds to Age, and Feature 2 corresponds to Body Mass Index (BMI).

Q4: Why are the numerical values in `diabetes_X` so small (e.g., between -0.1 and +0.1)?
Answer: Scikit-learn's built-in Diabetes dataset has been pre-standardized: each of the 10 feature columns has been mean-centered to 0.0 and scaled by standard deviation times $\sqrt{N}$.

Q5: What does `regr.coef_` represent in this program?
Answer: An array of 2 floating-point values representing the partial regression slope coefficients: β_1 for Age and β_2 for BMI.

Q6: What is the clinical interpretation of the positive coefficient for BMI ($\approx +605$)?
Answer: Holding age constant, an increase in BMI is strongly correlated with an increase in disease progression, demonstrating that BMI is a positive risk indicator.

Q7: What does the intercept `regr.intercept_` (≈ 152.84) signify?
Answer: It represents the expected progression score when both standardized features (Age and BMI) equal zero (their average baseline values).

Q8: Define Mean Squared Error (MSE) and state its units in this problem.
Answer: MSE is the average of the squared prediction errors: $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$. Its unit is $(\text{progression score})^2$.

Q9: How do you obtain Root Mean Squared Error (RMSE) and why is it useful?
Answer: $\text{RMSE} = \sqrt{\text{MSE}} \approx \sqrt{2548.07} \approx 50.48$. It is useful because its unit matches the target variable (points of disease progression), making it clinically interpretable.

Q10: What is the theoretical maximum and minimum value of R^2?
Answer: The maximum is $+1.0$ (perfect prediction). It has no theoretical lower bound; it can be negative $(-\infty, 1.0]$ when evaluated on unseen test data.

Q11: In Figure 2, what does the dashed red line `'r--'` represent?
Answer: It represents the ideal line of equality ($y = \hat{y}$). If the model had an $R^2 = 1.0$, every plotted point would lie exactly on this line.

Q12: What does `plt.subplot(1, 2, 2)` specify?
Answer: A subplot grid of 1 row and 2 columns, activating the 2nd panel (the right subplot).

Q13: Why is `alpha=0.7` used in the scatter plot calls?
Answer: It sets marker opacity to 70%, allowing overlapping data points to remain visible rather than obscuring each other.

Q14: What is multicollinearity and how does it affect multiple linear regression?
Answer: Multicollinearity occurs when two or more predictor features are highly correlated with each other. It destabilizes coefficient estimation and makes individual slope interpretations unreliable.

Q15: How would you prevent overfitting in linear regression if you used all 10 features?
Answer: By applying regularization techniques such as Ridge Regression (L_2 penalty) or Lasso Regression (L_1 penalty / feature selection).

Q16: How is this program different from all the other 5 programs in Exercises 05–10?
Answer: It is the only continuous regression program (evaluates continuous target y via MSE and R^2), whereas Exercises 05–08 are classification and Exercise 09 is clustering.

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. Why is Exercise 10 a Regression problem rather than Classification?
Answer: The target variable is a continuous quantitative physiological score (diabetes disease progression from 25 to 346), not a discrete category.

2. What is the difference between Mean Squared Error (MSE) and R^2 Score?
Answer: MSE measures average squared residual error in squared target units (≈ 2548). R^2 measures the proportion of target variance explained by features ($0.42 = 42\%$).

3. What features are extracted by `diabetes_X[:, [0, 2]]`?
Answer: Feature index 0 corresponds to Age, and Feature index 2 corresponds to Body Mass Index (BMI).

4. What does the dashed red line `'r--'` represent in Figure 2?
Answer: The ideal prediction line of equality ($y = \hat{y}$). Points closer to this diagonal represent more accurate model predictions.

5. Why is slicing `[:-20]` and `[-20:]` used instead of `train_test_split()`?
Answer: Slicing deterministically splits the first 422 samples for training and the last 20 samples for testing without shuffling, following scikit-learn's standard documentation benchmark format.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	How to Fix / Debug
Using Classification Metrics (e.g. `accuracy_score`)	Attempting to calculate accuracy percentage or confusion matrix on continuous floating-point predictions.	Target is continuous. Always use regression metrics: `mean_squared_error` and `r2_score`.
Incorrect 1D feature slicing: `diabetes_X[:, 0, 2]`	Passing indices without list brackets produces an `IndexError: too many indices for array`.	Use double brackets for column selection: `diabetes_X[:, [0, 2]]`.
Confusing `[:-20]` and `[-20:]` slicing	Swapping train and test slices, training on only 20 samples and testing on 422.	Remember: `[:-20]` is train (first 422 rows); `[-20:]` is test (last 20 rows).
Missing `plt.show()` between figures	Calling `plt.figure()` twice without `plt.show()` can cause figures to render on top of each other or not display at all.	Place `plt.show()` after Figure 1 and another `plt.show()` after Figure 2.
Interpreting high MSE as a code error	Seeing $\text{MSE} \approx 2548$ and assuming the program has a bug.	MSE is in squared units (y^2). Since diabetes target scores range from 25 to 346, $\text{RMSE} = \sqrt{2548} \approx 50$ points, which is normal for this baseline dataset.

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Eight-Step Coding Flow Mnemonic: "L-S-S-M-F-P-E-P"

- **L — Load:** `diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)`
- **S — Select:** `diabetes_X = diabetes_X[:, [0, 2]]`
- **S — Slice:** `X_train, X_test = X[:-20], X[-20:]` & `y_train, y_test = y[:-20], y[-20:]`
- **M — Model:** `regr = linear_model.LinearRegression()`
- **F — Fit:** `regr.fit(diabetes_X_train, diabetes_y_train)`
- **P — Predict:** `diabetes_y_pred = regr.predict(diabetes_X_test)`
- **E — Evaluate:** Print `coef_`, `intercept_`, `MSE`, and `R^2`
- **P — Plot:** Subplots (Age vs Target & BMI vs Target) + Diagnostic (Actual vs Pred Fit Line)

12.2 Memorizing Evaluation Formulas for Lab Exam

> • **Regression Line:** $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$
> • **Mean Squared Error:** $\text{MSE} = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$
> • **R^2 Score:** $R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5`, `random_state=42`, `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line